Kecerdasan Buatan

Penyelesaian Masalah dengan Pencarian

Outline

- Pendahuluan
- Definisi Pencarian
- Pencarian Berbasis *Tree* (Pohon)
- Pencarian *Graph*
- Klasifikasi Metode Pencarian
- Metode Pencarian *Uninformed*
- Kinerja Pencarian *Uninformed*
- Tugas

Pendahuluan

- Aspek penting dari kecerdasan adalah penyelesaian masalah berbasis tujuan (*goal-based problem solving*)
- Banyak masalah dapat dirumuskan sebagai pencarian sederetan aksi yang mengarah ke tujuan
- Komponen pencarian:
  - Himpunan Status
  - Satu atau lebih Operator
  - Status mulai/awal
  - Ujian untuk memeriksa status tujuan

Masalah yang Baik

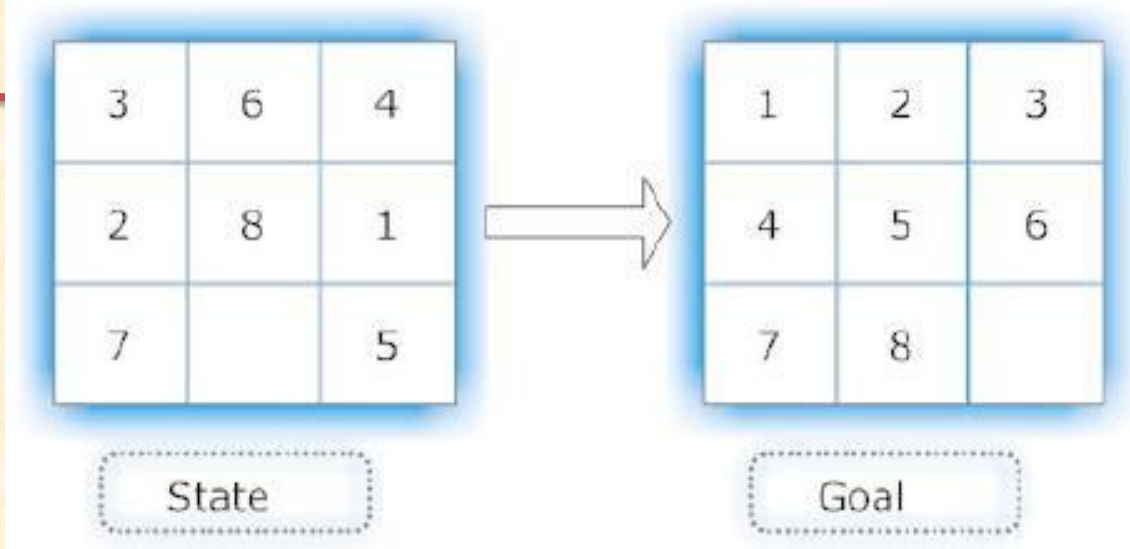
- Dapat dideskripsikan dengan:
  - initial state (status awal),
  - operator/fungsi *successor*, fungsi yang dapat memindahan status x ke suatu status baru s(x).

    f(x) → s(x)
  - state space (ruang status), semua status yang dapat dicapai dari initial state setelah diberlakukan fungsi f(x)
  - path (jalur), urutan melalui ruang status
  - *path cost* (biaya jalur), jumlah biaya dari setiap aksi sepanjang jalur
  - *goal test* (uji tujuan), sudahkah berada di tujuan?

Definisi Pencarian

- Pemeriksaan sistematis dari status-status untuk menemukan suatu jalur dari status awal ke status tujuan.
- *Search space* (ruang pencarian) terdiri dari himpunan status yang mungkin, dan operator yang mendefinisikan sambungannya.
- **Solusi**: Jalur dari status awal ke status yang memenuhi uji tujuan.
- Ada 3 kelas teknik/metode:
  - Menemukan suatu jalur awal → tujuan
  - Menemukan jalur terbaik
  - Buntu, tidak mendapatkan tujuan

Contoh: 8-Puzzle

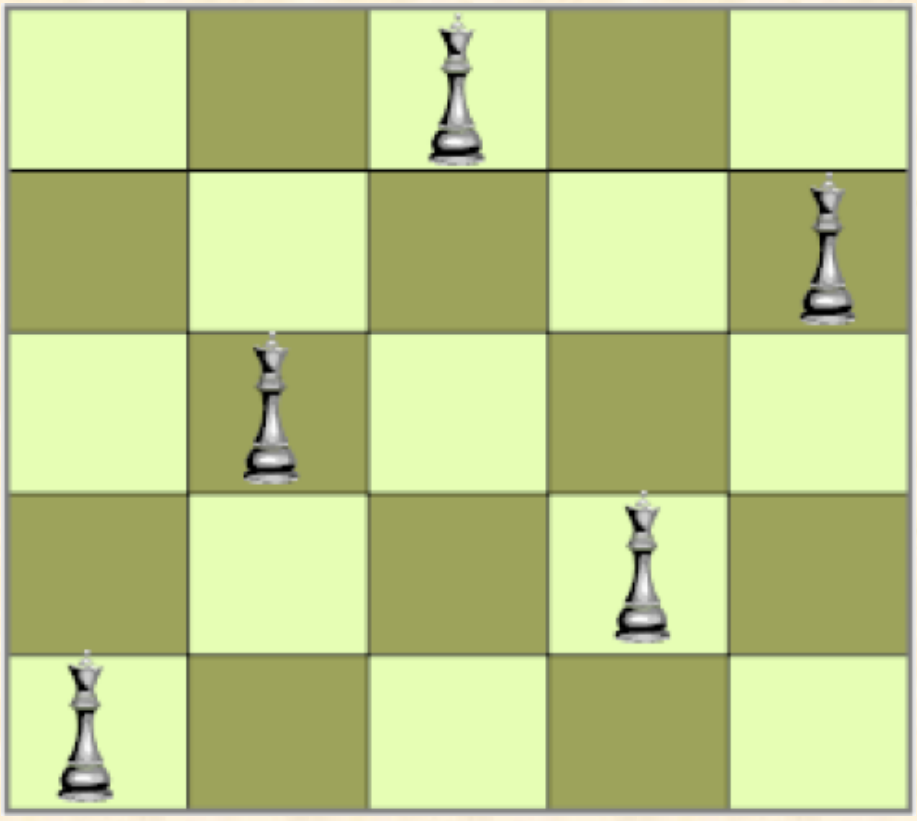


- *Status*: lokasi ubin kosong dan lokasi 8 ubin bernomor
- *Operator (successor)*: ubin kosong bergerak ke kiri, kanan, atas dan bawah
- *Tujuan*: cocokkan status yang diperoleh dengan status tujuan (goal)
- *Biaya Jalur*: setiap langkah biayanya 1; biaya total sesuai panjang jalur.

Contoh: N-Ratu

Mengatur N Ratu pada papan catur N × N sehingga ratu-ratu tersebut tidak saling memakan.

- *Status:* 0 s.d N Ratu yang diatur pada papan catur
- *Operator (successor)*: tempatkan sebuah Ratu pada suatu kotak kosong
- *Goal (Tujuan)*: cocokkan status dengan N Ratu pada papan catur dan tidak saling makan
- *Biaya jalur*: 0.



Contoh: 3 Cewek dan 3 Kanibal

- 3 cewek dan 3 kanibal ada di sisi kiri sungai.
- Semuanya harus menyeberang ke sisi kanan sungai, menggunakan *boat* yang hanya mampu dinaiki 2 orang.
- Jumlah kanibal tidak boleh melebihi jumlah cewek, di sisi sungai, kapanpun.
- Bagaimana agar semuanya dapat menyeberang?
- Status:
  - Jumlah cewek di sisi kiri
  - Jumlah kanibal di sisi kiri
  - Di sisi mana *boat* berada.

Contoh: 3 Cewek dan 3 Kanibal

Contoh: Status Awal: (3, 3, kiri)

- *Operator*: Suatu perpindahan yang diwakili oleh jumlah cewek dan jumlah kanibal dalam *boat* pada satu waktu. Ada 5 kemungkinan:
  - (2 cewek, 0 kanibal)
  - (1 cewek, 0 kanibal)
  - (1 cewek, 1 kanibal)
  - (0 cewek, 1 kanibal)
  - (0 cewek, 2 kanibal)
- *Goal (Tujuan)*: (3, 3, kanan)
- *Biaya jalur*: jumlah penyeberangan.

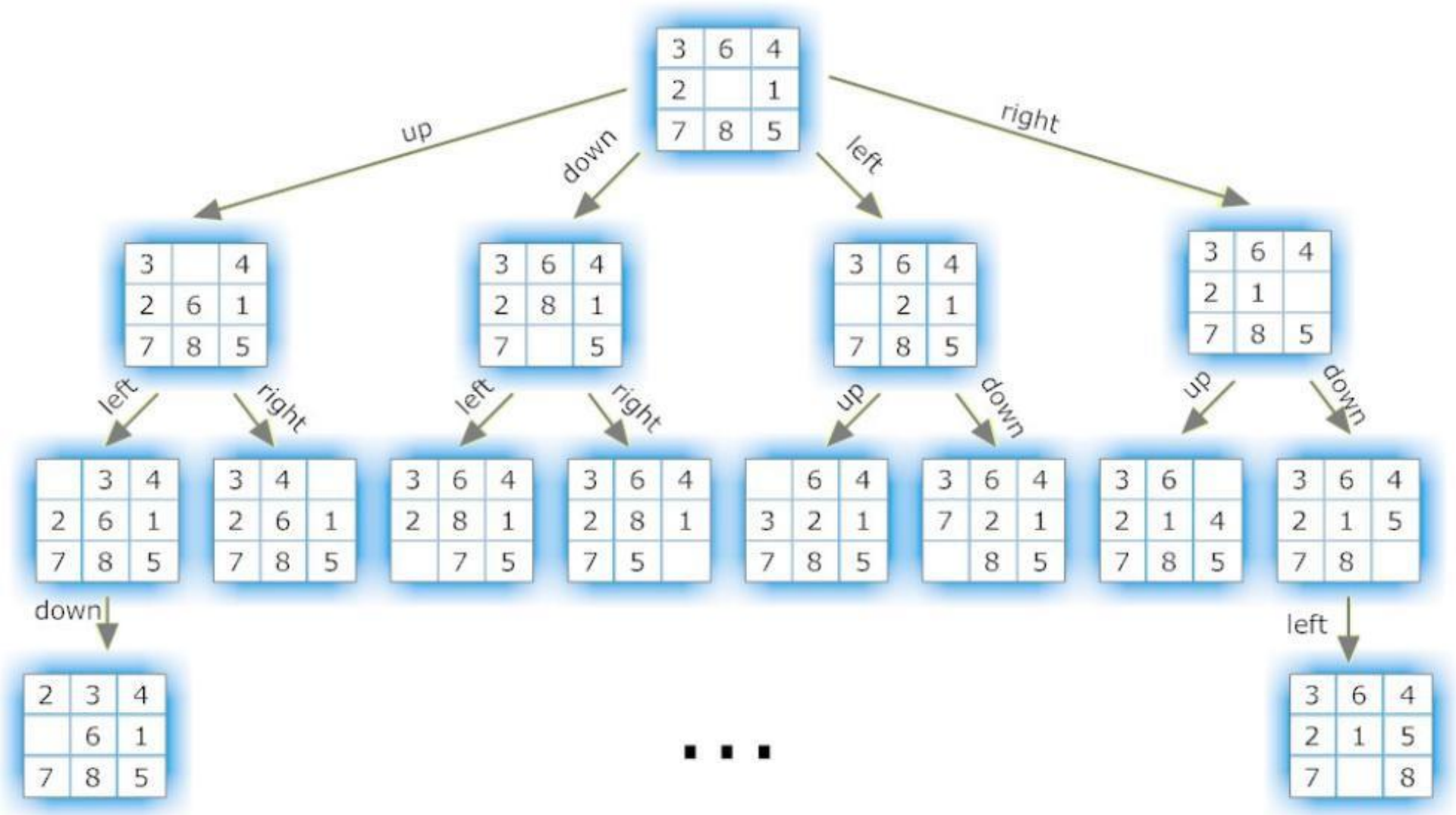
Pencarian Berbasis Pohon

- Himpunan semua jalur dalam ruang status dapat digambarkan sebagai graf *node-node* yang terhubung.
- Jejak jelajah pencarian dapat membentuk *tree* (pohon)
- Istilah penting:
  - *Root node (akar)*: mewakili node awal pencarian;
  - *Leaf node (daun)*: *node* berhenti, tanpa anak;
  - *Ancestor/descendant*: *node* A adalah *ancestor* B jika A adalah induk B atau A adalah nenek dari induknya B. Jika A adalah *ancestor* B, maka B dikatakan *descendant* (keturunan) dari A;
  - *Branching factor*: jumlah anak maksimum dari suatu node daun dalam pohon pencarian;
  - *Path (jalur)*: jalur dalam pohon pencarian yang mewakili jalur lengkap jika itu bermula dengan node awal dan berakhir dengan node tujuan. Jika tidak, disebut jalur *partial*.

Pencarian Berbasis Pohon

- Suatu node dapat ditampilkan oleh struktur data berikut:
  - Suatu deskripsi status;
  - Suatu pointer ke induk dari node;
  - Kedalaman dari node;
  - Operator yang membangkitkan node ini;
  - Biaya jalur (jumlah biaya operator) diperoleh dari status awal (mulai).
- **Kerugian**: dapat secara berulang mengunjungi node yang sama.
- **Solusi**: simpan semua node yang telah dikunjungi tetapi perlu tambahan memory.

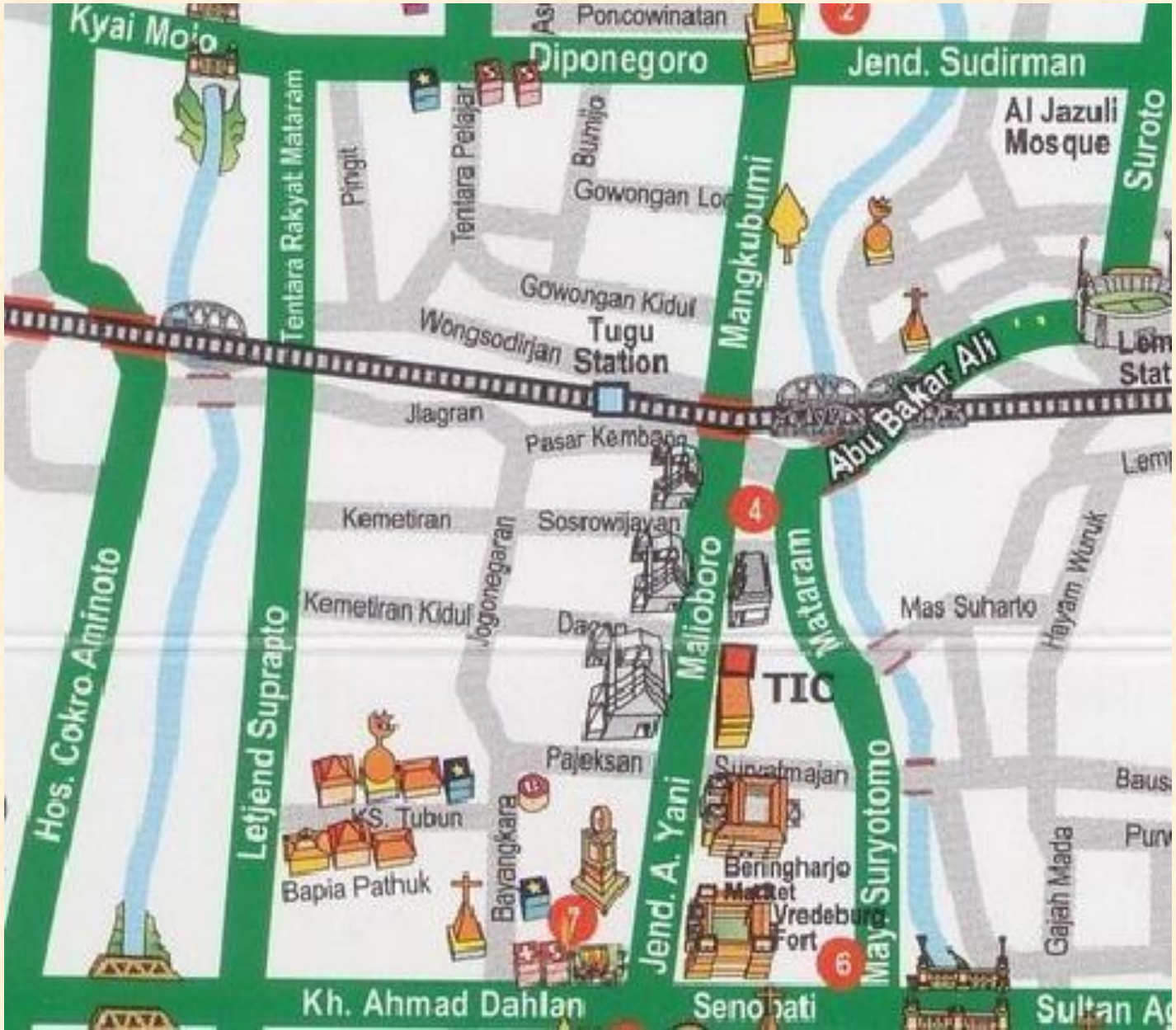
Contoh


up
down
left
right
left
right
left
right
up
down
up
down
down
left
. . .

Pencarian Berbasis Graf

- Ruang status diwakili oleh graf G(V,E), dimana V adalah himpunan node dan E adalah himpunan vertex dari satu node ke node lain.
- Informasi pada tiap node:
  - Suatu deskripsi status;
  - Induk dari node;
  - Operator yang membangkitkan node tersebut dari induknya;
  - Informasi lain.

Contoh: Peta Yogyakarta



Penyesuaian bagi Graf

- ***Status awal***: Node yang ditunjuk sebagai node awal.
- ***Ruang status***: Awalnya, node awal S, ditulis V={S}. Kemudian S di*expand* dan node yang dibangkitkannya ditambahkan ke V dan E. Proses ini berlanjut sampai tujuan ditemukan;
- ***Jalur*** : setiap node mewakili solusi parsial dari node awal untuk node yang diberikan. Dari node ini ada banyak jalur yang mungkin yang mempunyai jalur parsial ini sebagai *prefix*-nya;
- ***Biaya jalur***: jumlah biaya vertex pada jalur solusi;
- ***Uji tujuan***: Uji diterapkan terhadap suatu status untuk menentukan jika node terkait adalah tujuan & memenuhi semua kondisi tujuan;
- ***Solusi***: sederetan operator yang dikaitkan dengan suatu jalur dalam ruang status dari node awal sampai node tujuan

Algoritma Pencarian Dasar

Pencarian_Umum(masalah, strategi)

Gunakan status awal dari masalah untuk mengawali pohon pencarian

Loop

If tidak ada node yang akan di*expand*

Then return GAGAL;

Berdasarkan pada strategi, pilih node untuk perluasan;

terapkan uji tujuan;

If node adalah status tujuan

then return SOLUSI

Else *expand* node tersebut dan tambahkan node-node yang dihasilkan dan vertex ke pohon pencarian.

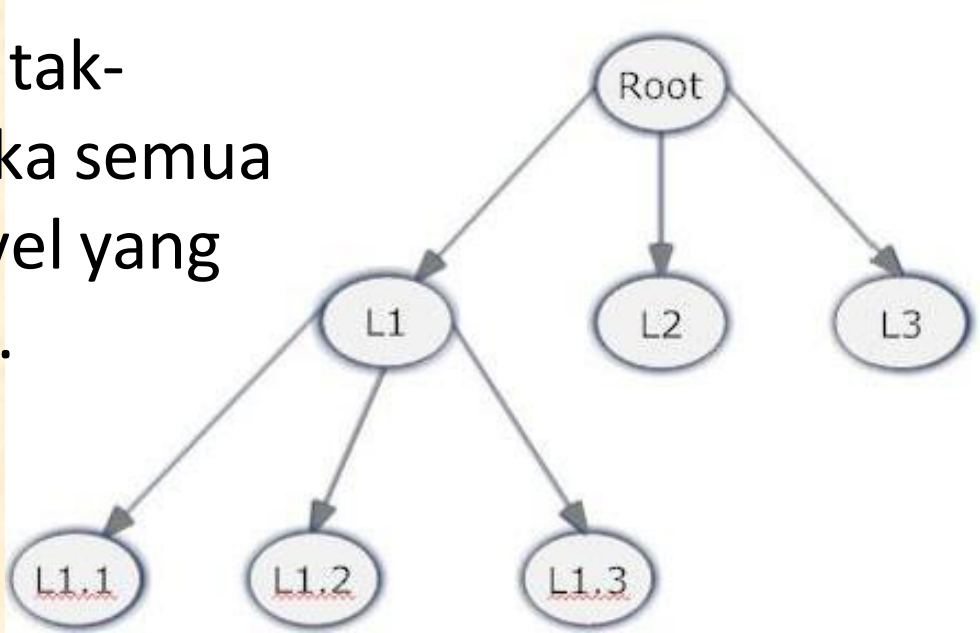
End;

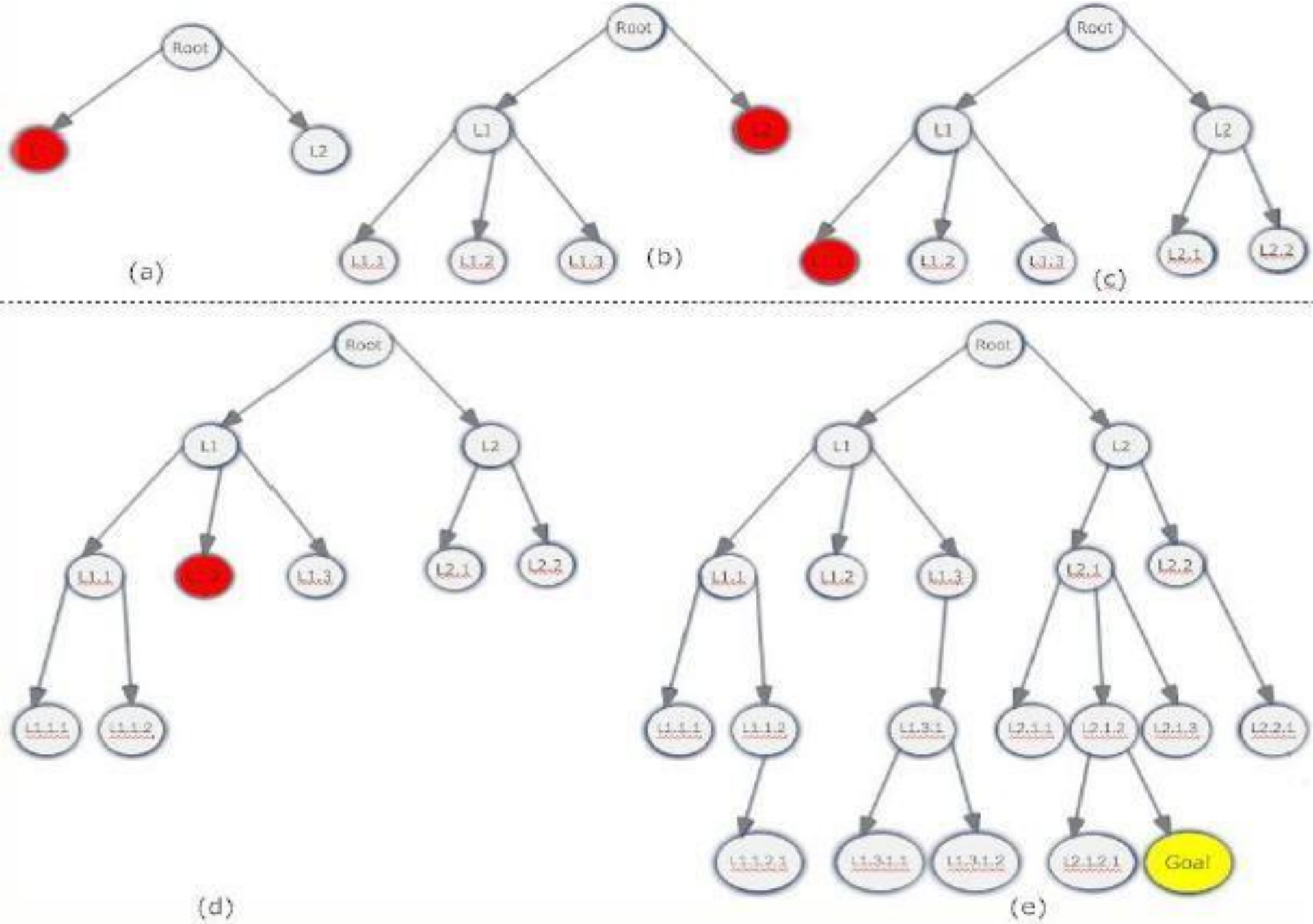
Klasifikasi Metode Pencarian

- 2 kategori utama:
  - *uninformed (blind) search;*
  - *informed (heuristic) search.*
- Contoh *Blind Search*:
  - *breadth-first;*
  - *depth-first;*
  - *iterative deepening depth-first;*
  - *bidirectional;*
  - *branch and bound (uniform cost search)*.
- Contoh *Heuristc Search*:
  - *hill climbing;*
  - *beam search*
  - *greedy;*
  - *best first search*
  - *heuristics*.

Breadth First Search (BFS)

- **Pencarian melebar lebih dulu**
- Setiap status pada layer/level yang diberikan di*expand* sebelum bergerak ke status-status pada layer berikutnya.
- Cocok untuk pohon yang tak-seimbang, tetapi boros jika semua node tujuan ada pada level yang dalam dan di sebelah kiri.




Root
L2
(a)
Root
L1
L1.1
L1.2
L1.3
(b)
Root
L1
L2
L1.2
L1.3
L2.1
L2.2
(c)
Root
L1
L2
L1.1
L1.3
L2.1
L2.2
L1.1.1
L1.1.2
(d)
Root
L1
L2
L1.1
L1.2
L1.3
L2.1
L2.2
L1.1.1
L1.1.2
L1.3.1
L2.1.1
L2.1.2
L2.1.3
L2.2.1
L1.1.2.1
L1.3.1.1
L1.3.1.2
L2.1.2.1
Goal
(e)

Algoritma BFS

Langkah 1. Bentuk antrian Q dan masukkan status awal (misal: Root).

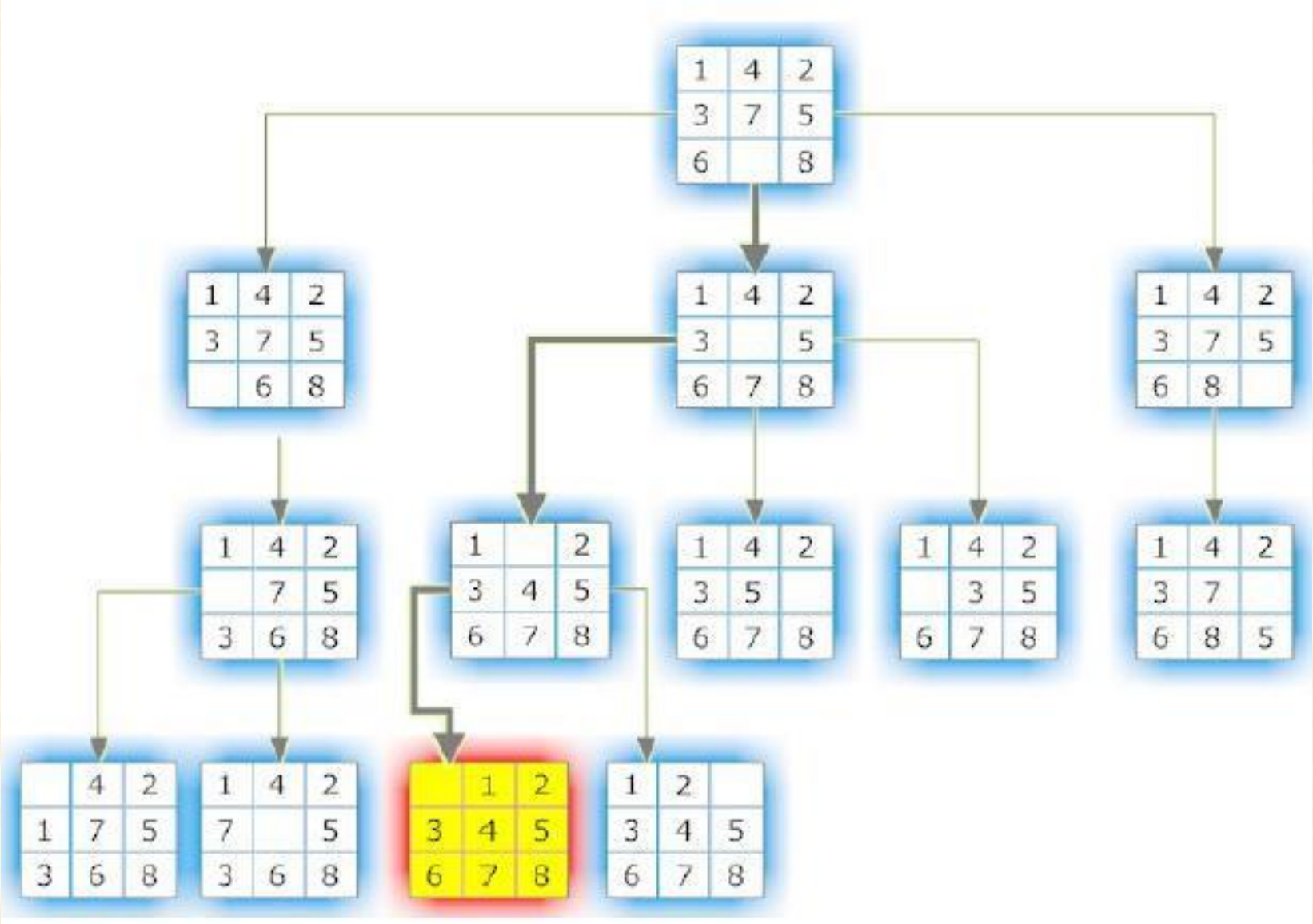
Langkah 2. Sampai Q kosong atau status tujuan ditemukan

do:

- **Langkah 2.1** Apakah elemen pertama dalam Q adalah tujuan?
- **Langkah 2.2** IF Iya, Selesai dan Return Status ini
- **Langkah 2.3** IF Tidak
  - **Langkah 2.3.1** Hapus elemen pertama dalam Q.
  - **Langkah 2.3.2** Bangkitkan status-status baru (*successor*).
  - **Langkah 2.3.3** Tambahkan status-status baru itu ke akhir Q.

Langkah 3. IF tujuan dicapai, SUKSES; else GAGAL.

Contoh: 8-Puzzle

- 

Contoh: 3 Cewek dan 3 Kanibal

- Status dapat berbentuk:

  (Kiri(#C, #K), Boat, Kanan(#C, #K)
- Dapat disederhanakan menjadi: (#C, #K, Kr/Kn)
- Ada 5 kemungkinan aksi dari suatu status:
  - Satu cewek bergerak ke sisi kanan;
  - Dua cewek bergerak ke sisi kanan;
  - Satu kanibal bergerak ke sisi kanan;
  - Dua kanibal bergerak ke sisi kanan;
  - Satu kanibal dan satu cewek bergerak ke sisi kanan.

Ruang Status

Sisi Kiri			
0C 0K Kr	1C 0K Kr	2C 0K Kr	3C 0K Kr
0C 1K Kr	1C 1K Kr	2C 1K Kr	3C 1K Kr
0C 2K Kr	1C 2K Kr	2C 2K Kr	3C 2K Kr
0C 3K Kr	1C 3K Kr	2C 3K Kr	3C 3K Kr

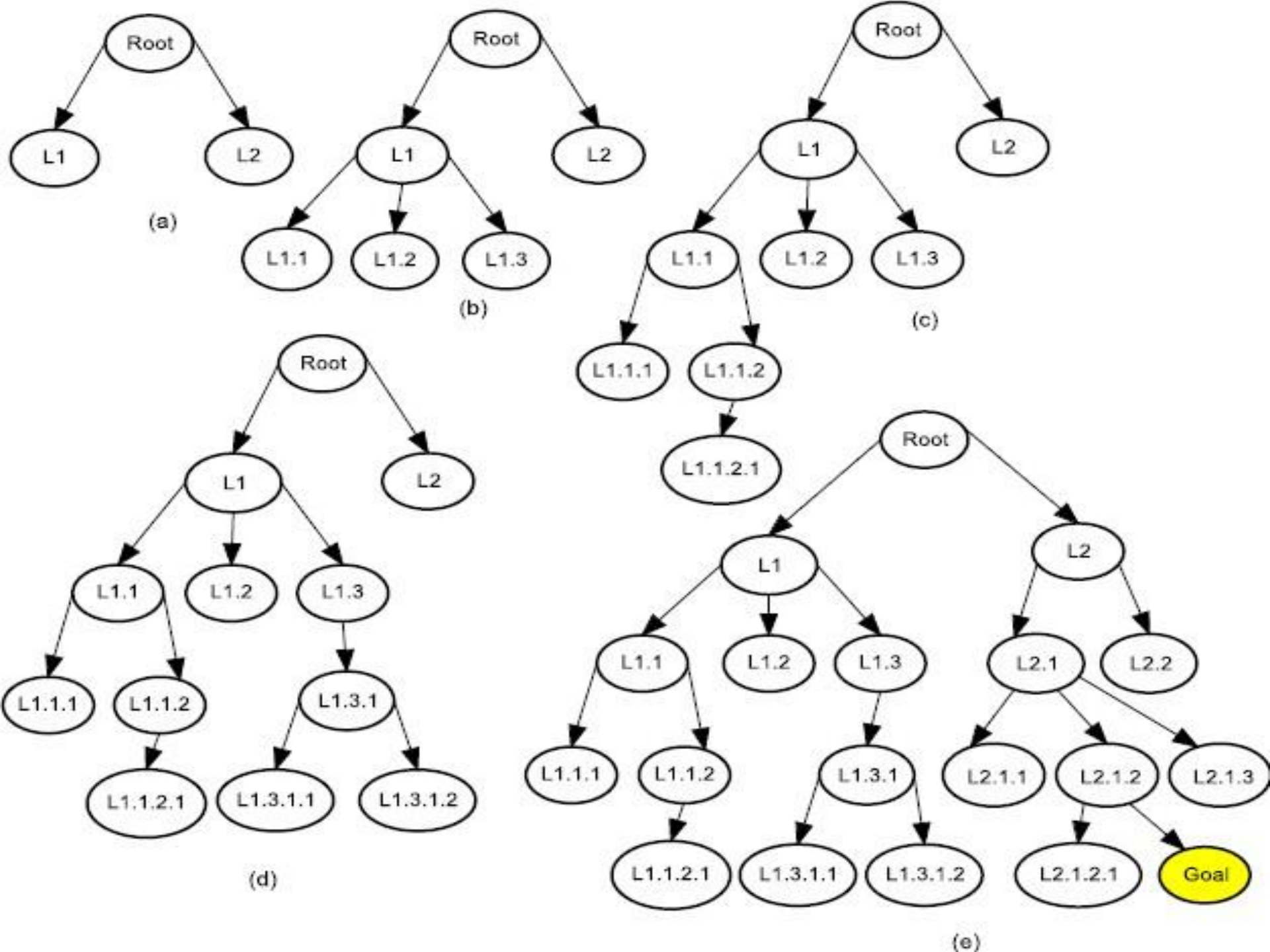
Sisi Kanan			
0C 0K Kn	1C 0K Kn	2C 0K Kn	3C 0K Kn
0C 1K Kn	1C 1K Kn	2C 1K Kn	3C 1K Kn
0C 2K Kn	1C 2K Kn	2C 2K Kn	3C 2K Kn
0C 3K Kn	1C 3K Kn	2C 3K Kn	3C 3K Kn

Solusi

Sisi Kiri	Boat	Sisi Kanan
3C 3K	Kr	0C 0K
3C 1K	Kn	0C 2K
3C 2K	Kr	0C 1K
3C 0K	Kn	0C 3K
3C 1K	Kr	0C 2K
1C 1K	Kn	2C 2K
2C 2K	Kr	1C 1K
0C 2K	Kn	3C 1K
0C 3K	Kr	3C 0K
0C 1K	Kn	3C 2K
0C 2K	Kr	3C 1K
0C 0K	Kn	3C 3K

Depth First Search (DFS)

- **Pencarian mendalam lebih dahulu**
- Mirip algoritma BFS, bedanya anak diletakkan di awal antrian (*queue*), bukan di akhir.
- Antrian dapat diganti dengan *stack*. Node-node di dalam *stack* di-**pop** dan node-node baru di-**push**.
- Strategi ini selalu meng*expand* node di level paling dalam pada pohon pencarian
- Jalur akan di*expand* terus sampai ditemukan tujuan atau tidak ada lagi jalur yang dapat di*expand*


Root
L1
L2
(a)
Root
L1
L2
L1.1
L1.2
L1.3
(b)
Root
L1
L2
L1.1
L1.2
L1.3
L1.1.1
L1.1.2
L1.1.2.1
(c)
Root
L1
L2
L1.1
L1.2
L1.3
L1.1.1
L1.1.2
L1.3.1
L1.1.2.1
L1.3.1.1
L1.3.1.2
(d)
Root
L1
L2
L1.1
L1.2
L1.3
L2.1
L2.2
L1.1.1
L1.1.2
L1.3.1
L2.1.1
L2.1.2
L2.1.3
L1.1.2.1
L1.3.1.1
L1.3.1.2
L2.1.2.1
Goal
(e)

Algoritma DFS

Langkah1. Bentuk antrian Q dan masukkan status awal (misal: Root).

Langkah 2. Sampai Q kosong atau status tujuan ditemukan

do:

Langkah 2.1 Apakah elemen pertama dalam Q merupakan tujuan?

Langkah 2.2 IF Iya, Selesai dan Return status ini

Langkah 2.3 IF bukan

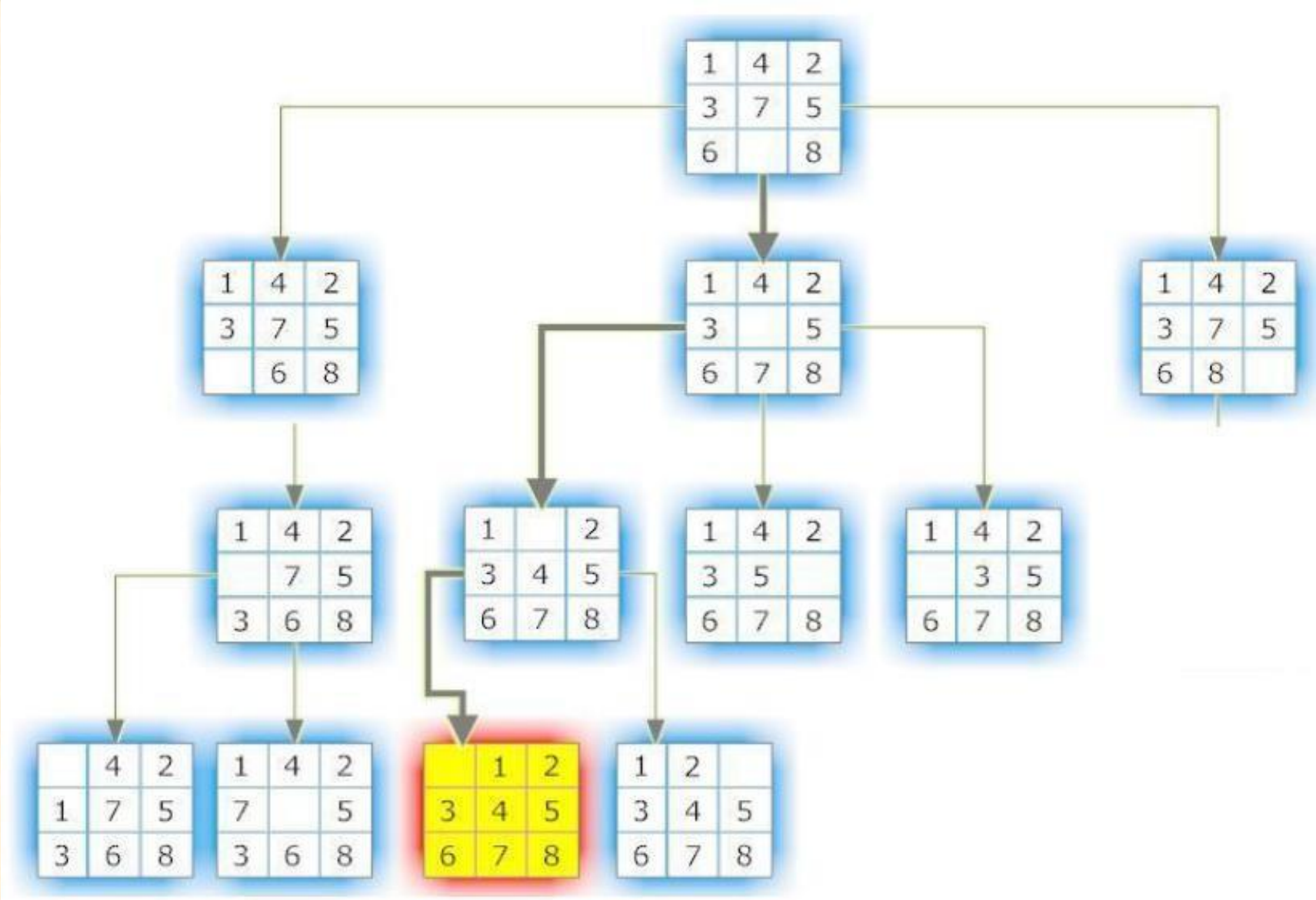
Langkah 2.3.1 Hapus elemen pertama dalam Q.

Langkah 2.3.2 Bangkitkan status-status baru (*successor*).

Langkah 2.3.3 Tambahkan status baru (urut terbaru) ke depan Q.

Langkah 3. IF tujuan dicapai, SUKSES; else GAGAL.

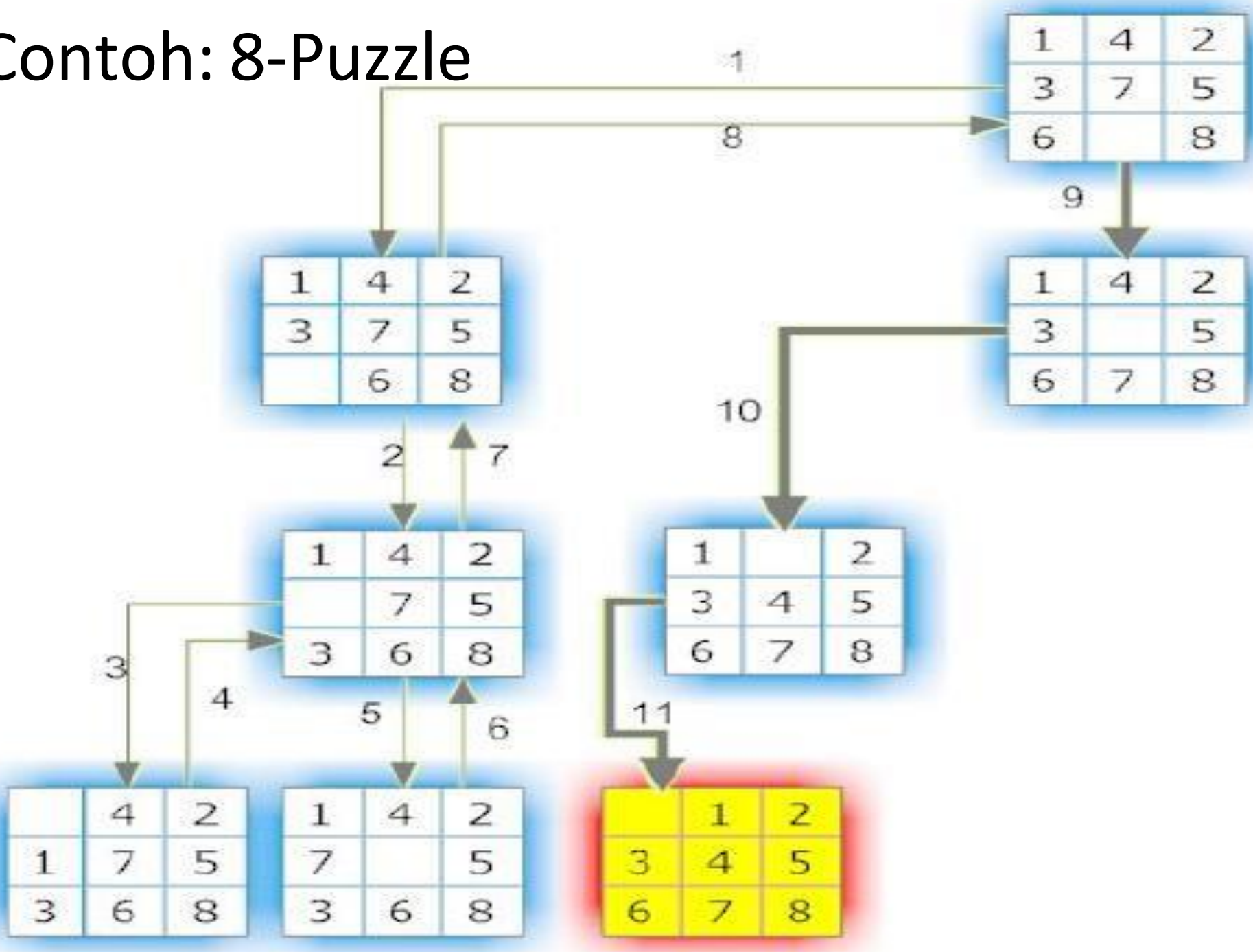
Contoh: 8-Puzzle



Pencarian *Backtracking* (Mundur)

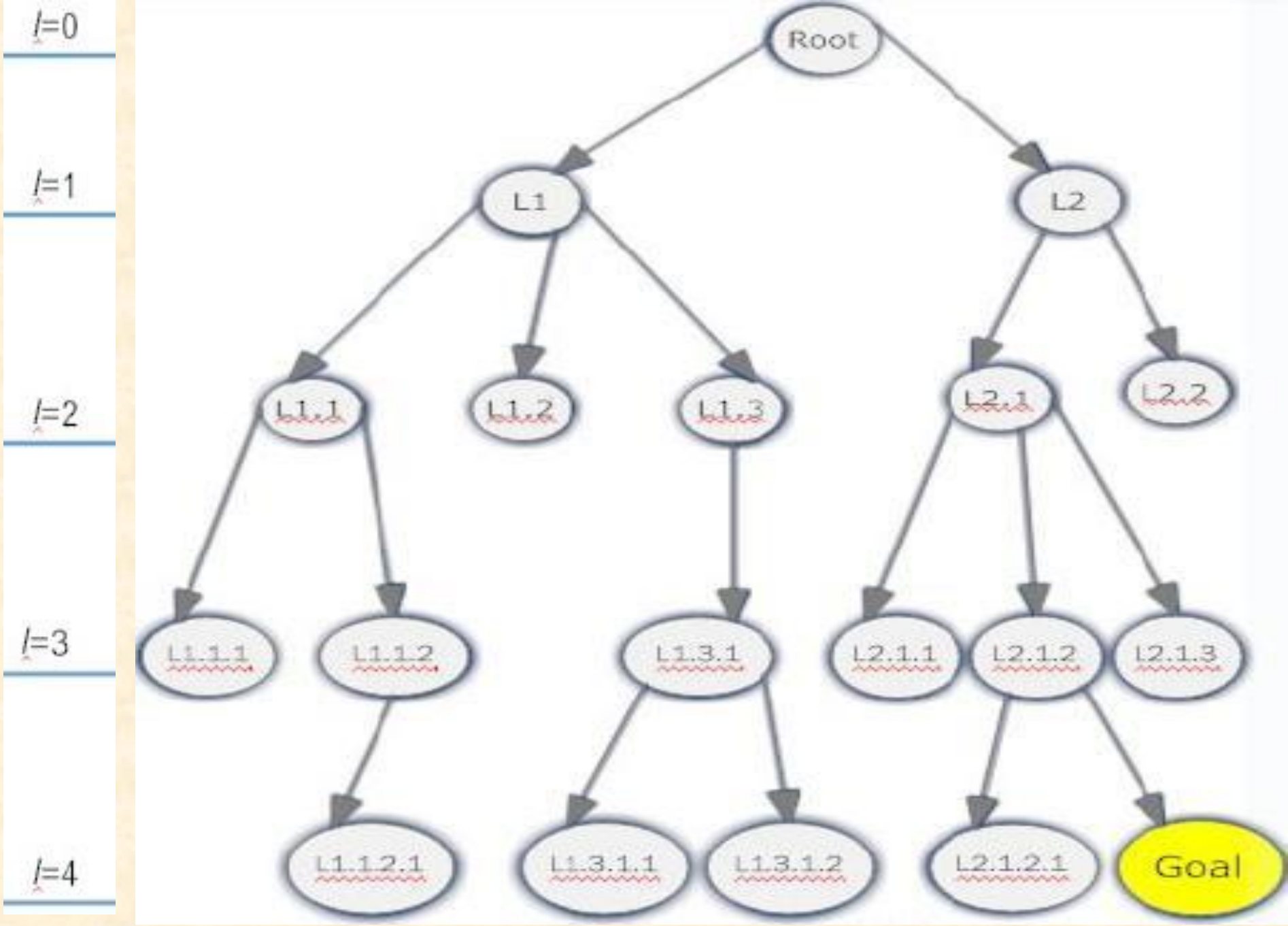
- Pencarian *depth-first* yang memilih nilai-nilai untuk satu variabel pada satu waktu dan mundur ketika suatu variabel sudah tidak punya nilai untuk diberikan.
- Lebih irit memory. Hanya satu *successor* yang dibangkitkan pada satu.
- Menyelidiki semua situasi yang mungkin dan mengembalikan semua solusi.
- Slide berikut adalah proses *backtracking* untuk solusi pertama masalah 8-Puzzle. Panah menunjukkan urutan status dijelajah dan diexpand.

Contoh: 8-Puzzle

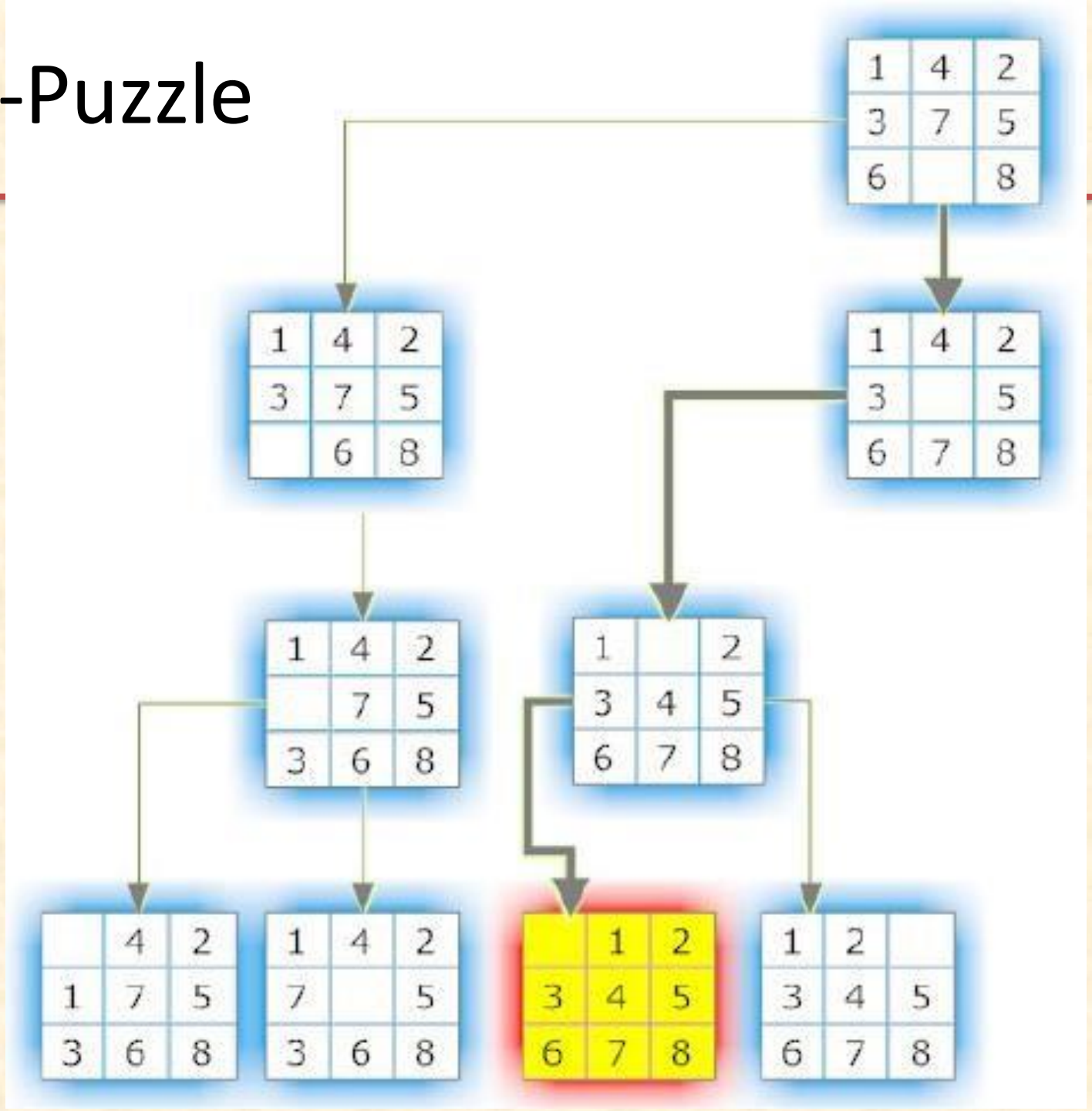


Pencarian *Depth Bounded*

- Dikenal juga sebagai *depth bounded* atau *depth limited* (kedalaman terbatas)
- Serupa DFS tetapi jalur yang panjangnya sudah mencapai level tertentu, l, tidak diperluas lagi.
- Dapat diimplementasi menggunakan *stack* atau antrian (node ditambahkan di depan). Node-node dengan kedalaman melebihi l tidak dimasukkan (ditambahkan).


l=0
l=1
l=2
l=3
l=4
Root
L1
L2
L1.1
L1.2
L1.3
L2.1
L2.2
L1.1.1
L1.1.2
L1.3.1
L2.1.1
L2.1.2
L2.1.3
L1.1.2.1
L1.3.1.1
L1.3.1.2
L2.1.2.1
Goal

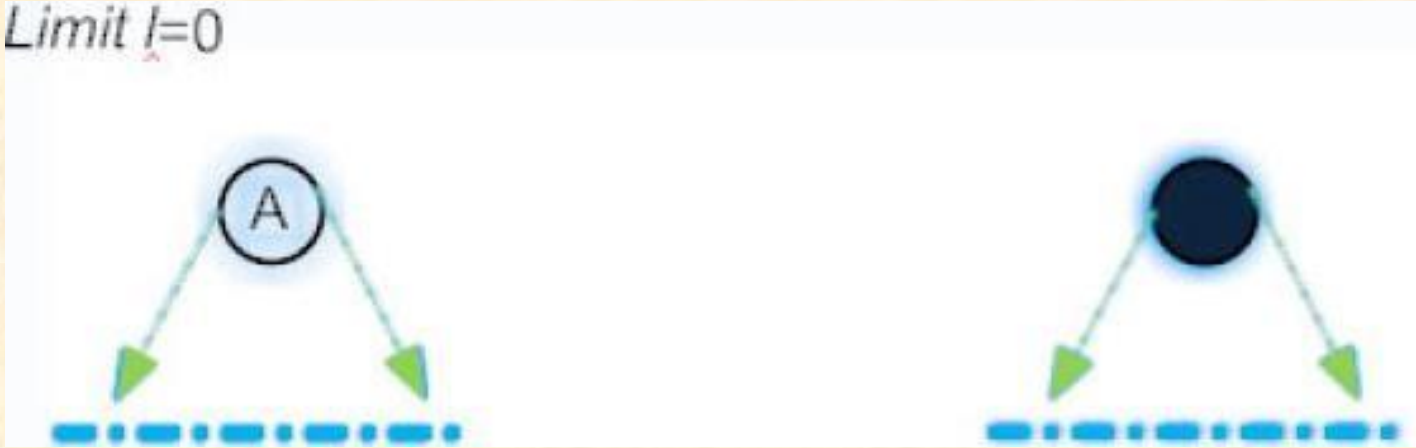
Contoh: 8-Puzzle

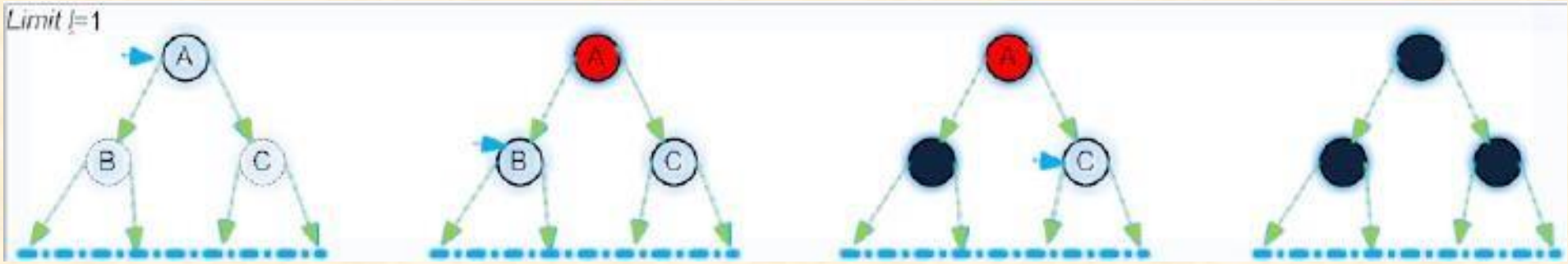


DFS diperdalam secara Iteratif

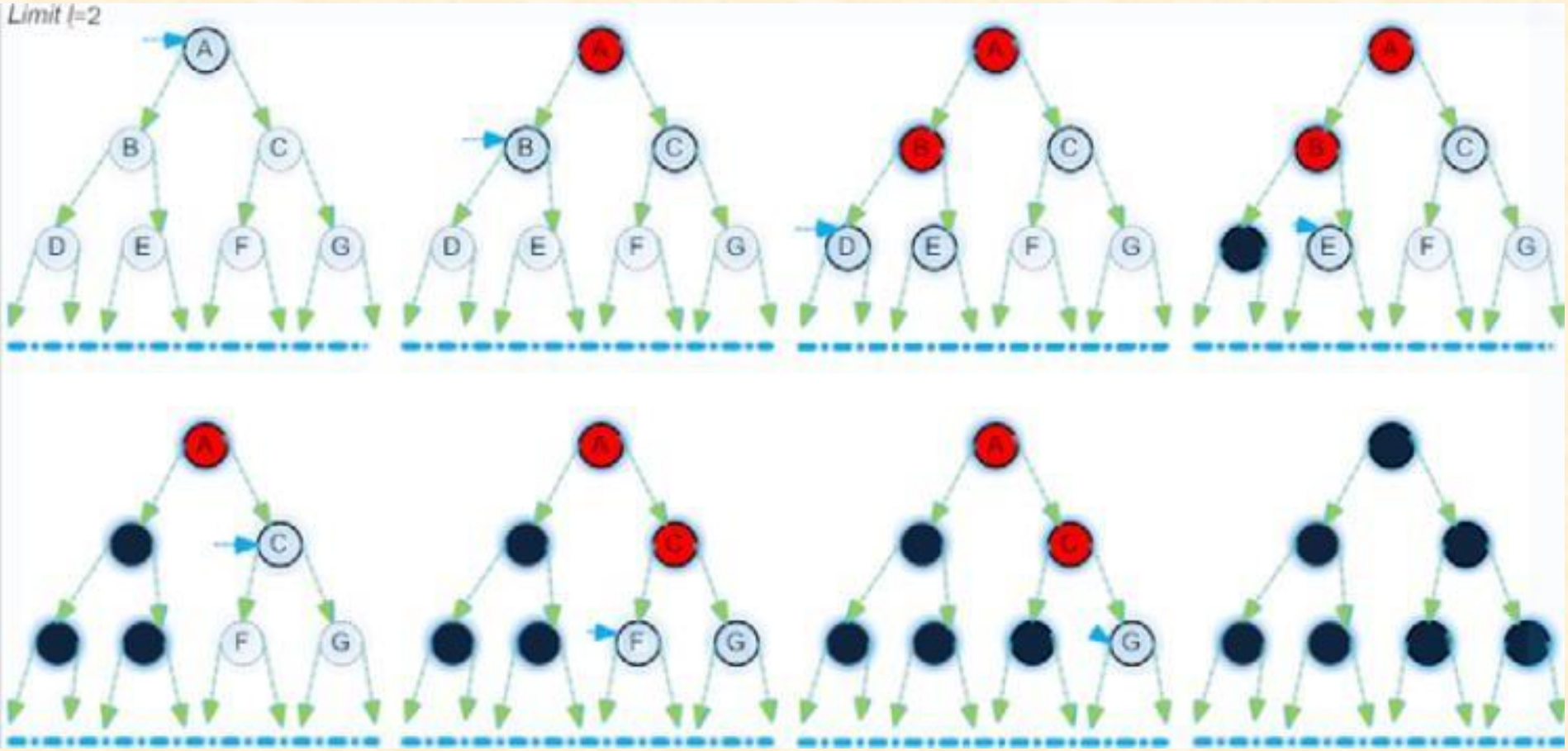
- Pencarian *iterative deepening dept bounded* DFS atau *iterative deepening.*
- Pencarian sampai level kedalaman terbatas yang dinaikkan secara bertahap sampai ditemukan tujuan.
- Pertama dicoba batas kedalaman l=0, jika tidak ditemukan solusi, dinaikkan l=1, kemudian l=2 dan seterusnya
- Berbasis pada pencarian *depth bounded/limited*. Pencarian *depth bounded* yang diulang sampai solusi ditemukan.

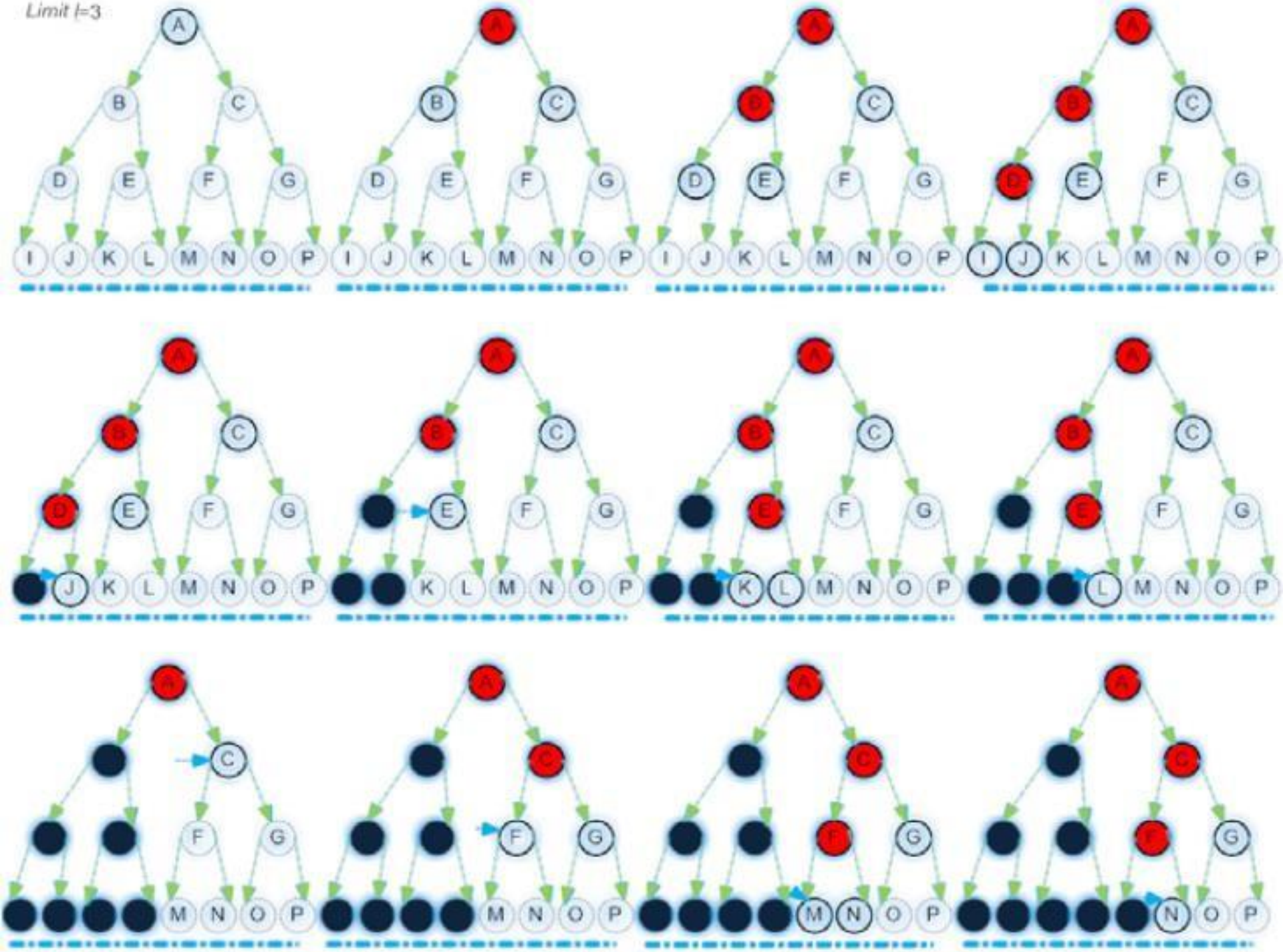
Contoh Pencarian DFS Iteratif





Contoh Pencarian DFS Iteratif




Limit l=3
A
B C
D E F G
I J K L M N O P

Algoritma DFS Kedalaman Iteratif

Kembalikan solusi atau gagal;

Input masalah;

l = 0

While tidak ada solusi, do

Terapkan DFS (masalah, kedalaman) dari status awal sampai ke dalam l

If tujuan ditemukan

Then Selesai dan Return Solusi,

Else naikkan batas kedalaman l=l+1 atau *inc*(l)

End.

Pencarian *Branch and Bound*
(Biaya Seragam)

- Node dengan biaya minimum selalu di*expand*.
- Begitu suatu jalur ke tujuan ditemukan, kemungkinan besar itulah jalur optimal.
- Ini digaransi dengan melanjutkan pembangkitan jalur-jalur parsial sampai semua jalur tersebut mempunyai biaya yang lebih atau sama dengan jalur yang ditemukan ke tujuan.

Algoritma Pencarian *Branch and Bound*

Kembalikan suatu solusi atau gagal

Q adalah antrian prioritas, diurutkan berdasarkan biaya terkini dari awal (*start*) ke tujuan (*goal*)

Langkah 1. Tambahkan status awal (*root*) ke Q.

Langkah 2. Sampai tujuan dicapai atau Q kosong

do

Langkah 2.1 Hapus jalur (*path*) pertama dari antrian Q;

Langkah 2.2 Buat jalur baru dengan memperluas jalur pertama ke semua tetangga dari node terminal.

Langkah 2.3 Hapus semua jalur yang ber-*loop*.

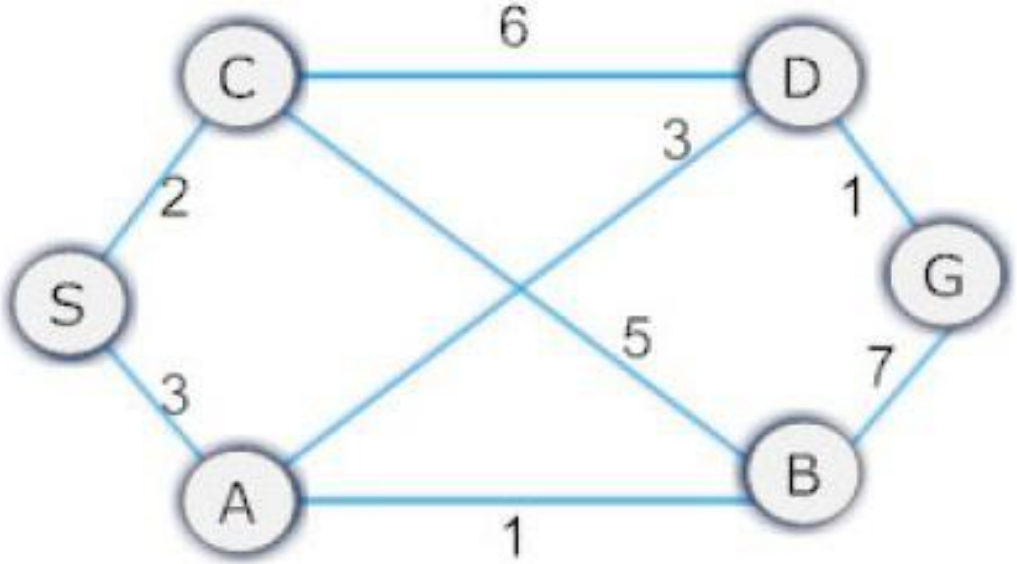
Langkah 2.4 Tambahkan jalur baru yang tersisa, jika ada, ke Q.

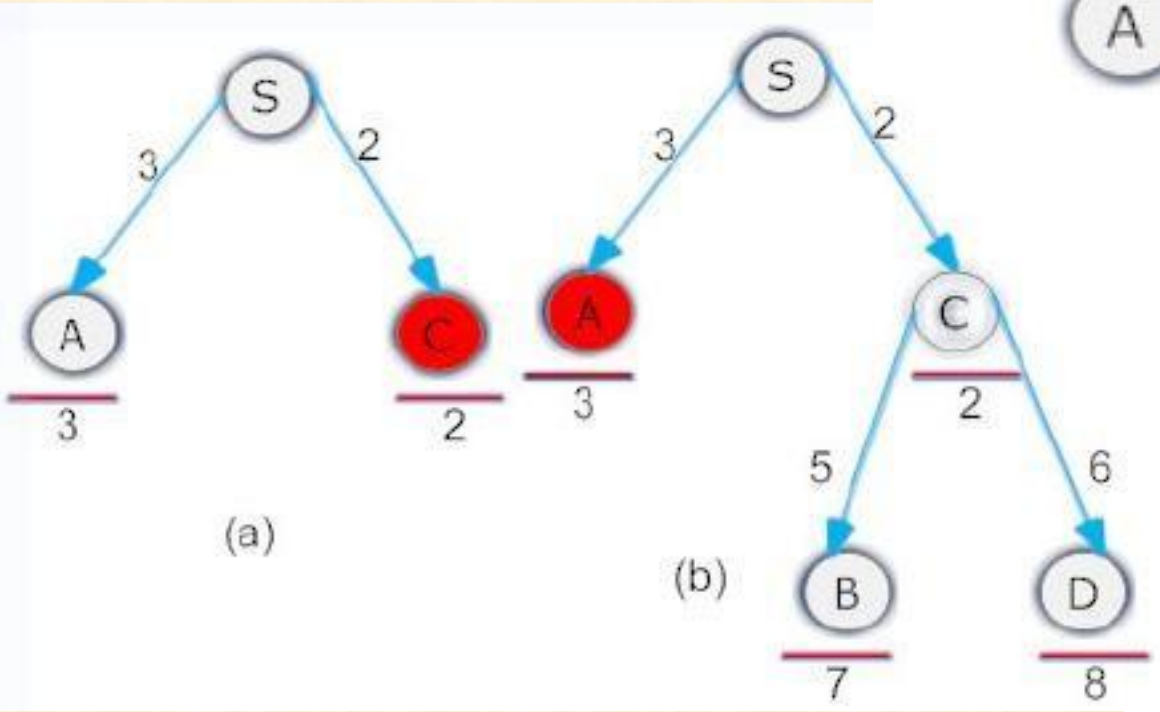
Langkah 2.5 Urutkan Q, jalur berbiaya murah ada di depan.

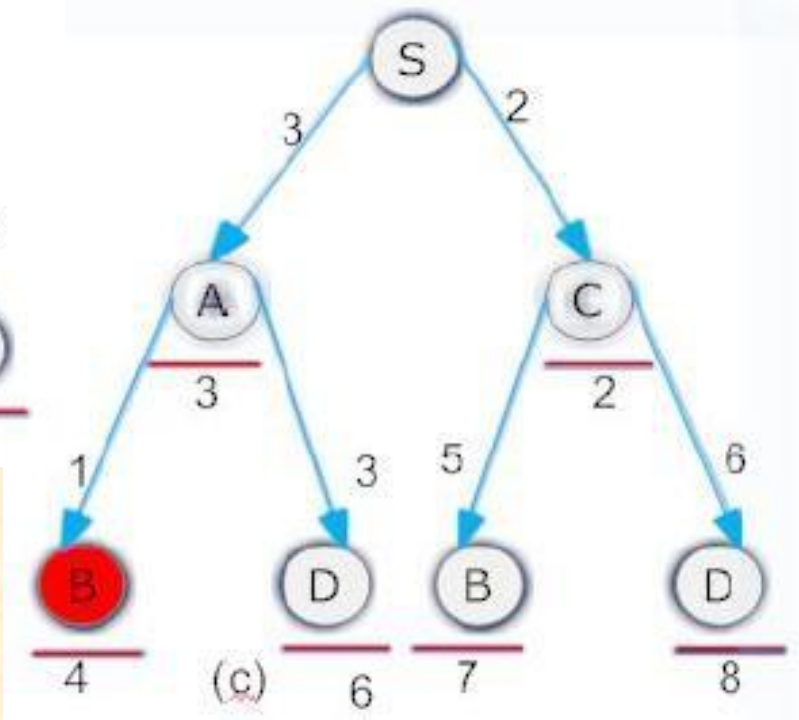
End

Contoh

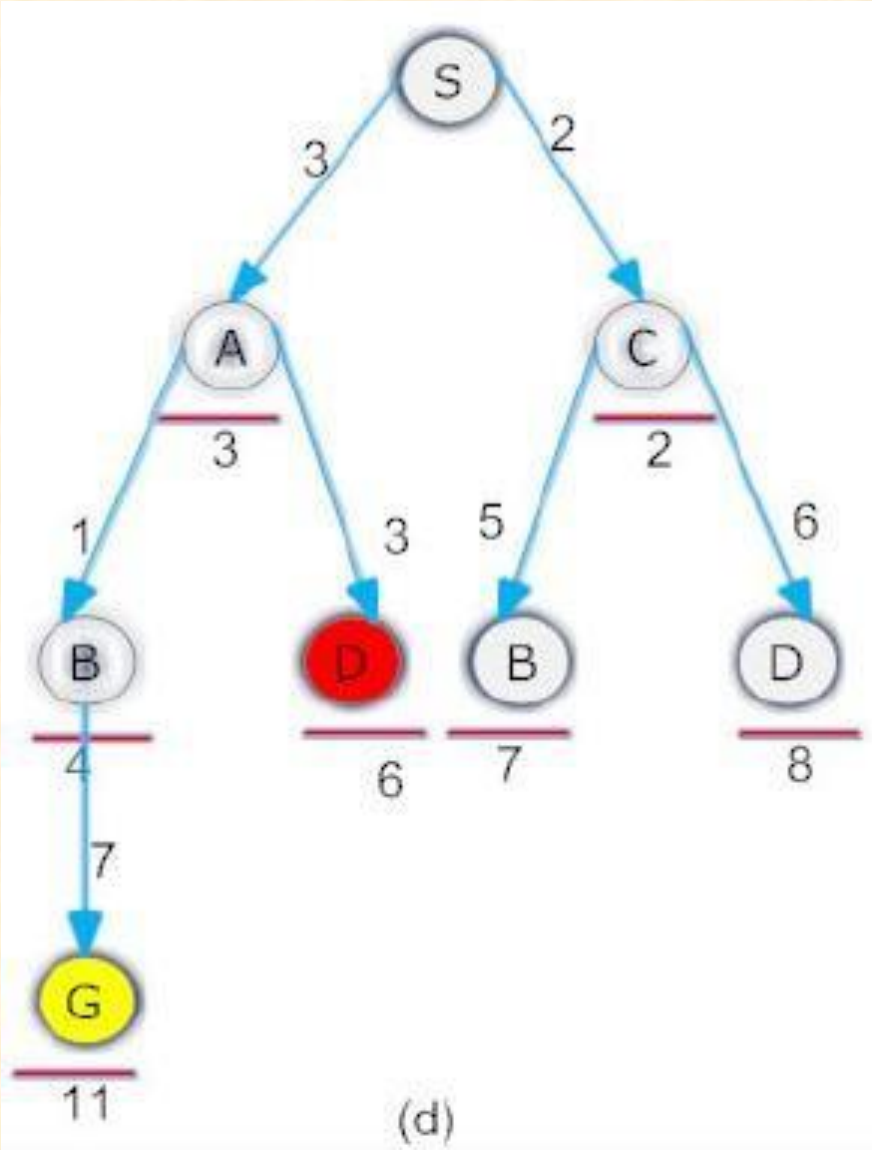
- Mana jalur terbaik?

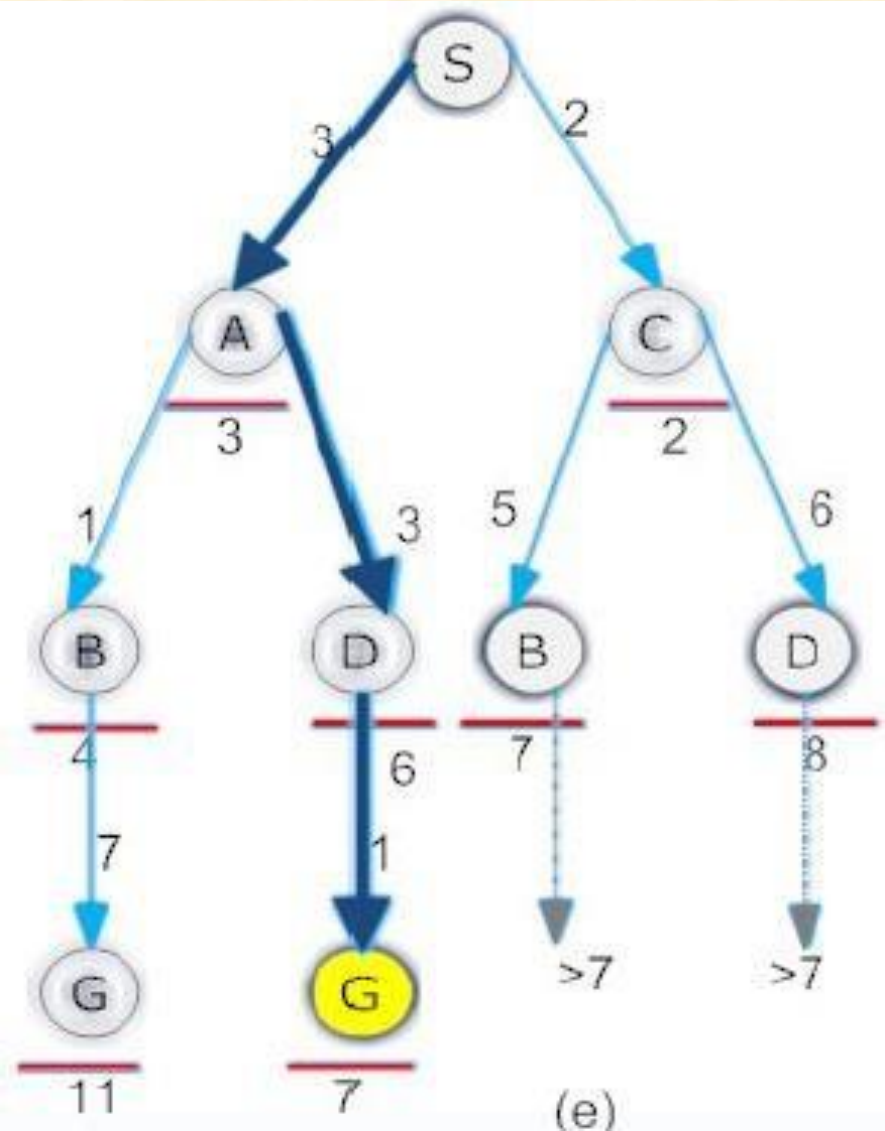






Contoh: Menentukan Jalur Terbaik

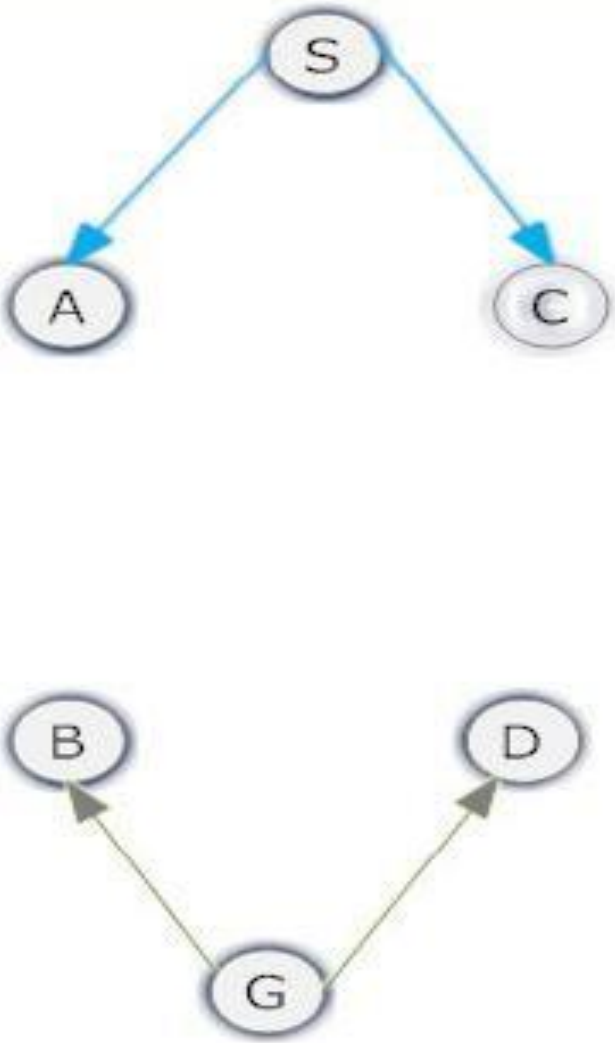




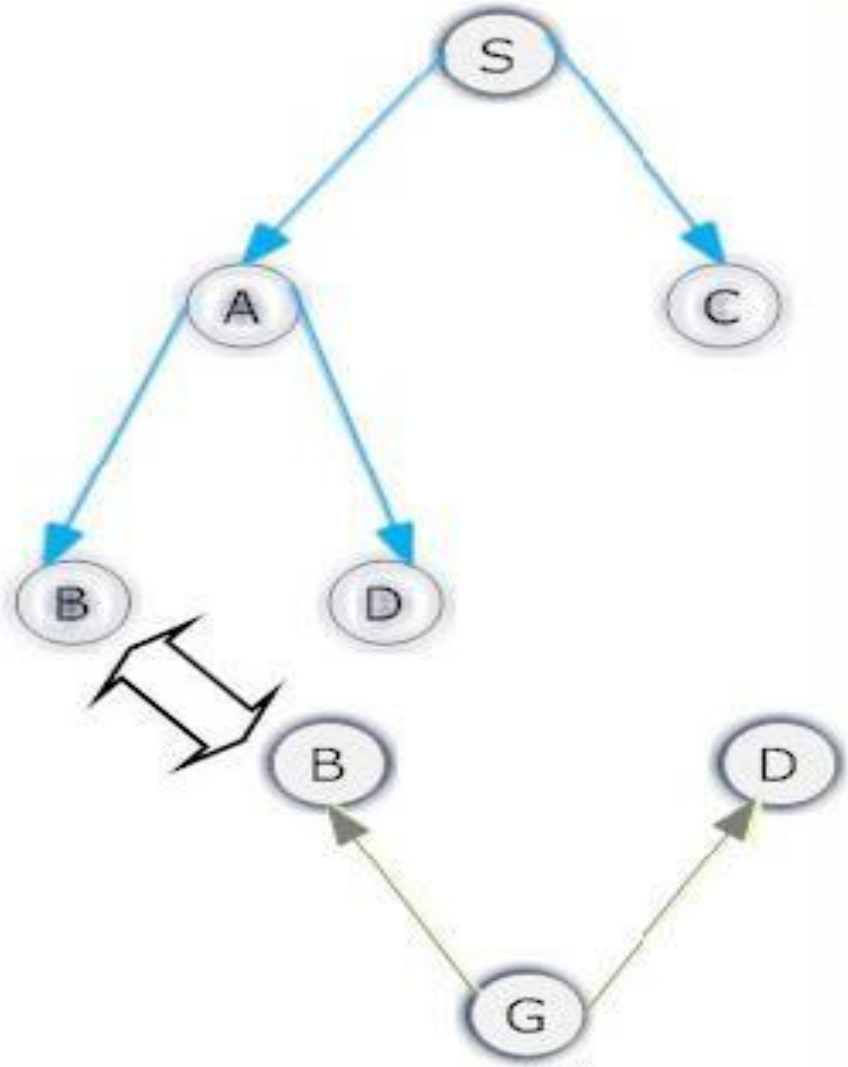
Pencarian Dua arah *(Bidirectional)*

- Ada 3 arah pencarian:
  - *forward* (maju);
  - *backward* (mundur);
  - *bidirectional* (dua arah)
- Pada pencarian dua arah, node-node di*expand* dari status awal ke tujuan secara bersamaan (*simultan*)
- Pada setiap tahap dicek apakah node-node yang dijumpai sudah dibangkitkan oleh yang lain.
- Jika Iya, maka gabungan jalur adalah solusinya.
- Dua pencarian dapat bekerja paralel: satu dari asal ke tujuan, satu dari tujuan ke awal
- Saat keduanya bertemu, diperoleh jalur yang baik.

Contoh: Jalur Terbaik?



(a)



(b)

Kinerja *Uninformed Search*

- Berbagai strategi pencarian dapat dibandingkan dalam hal:
  - (i) kesempurnaan (*completeness*): jaminan mendapatkan solusi;
  - (ii) kompleksitas waktu (*time*): berapa lama solusi ditemukan;
  - (iii) kompleksitas ruang (*space*): berapa banyak ruang (memory) digunakan.
  - (iv) Optimalitas(kualitas solusi): apakah solusi yang ditemukan bernilai optimal? Apakah biayanya minimal?.

Kinerja *Uninformed Search*

- Kompleksitas waktu dan ruang dipengaruhi oleh:
  - b: *branching factor* maksimum dari pohon pencarian;
  - d: kedalaman dari solusi paling-irit biaya;
  - m: kedalaman maksimum dari ruang status (mungkin ∞);
  - l: batas kedalaman bagi pencarian kedalaman terbatas (*limited search*).

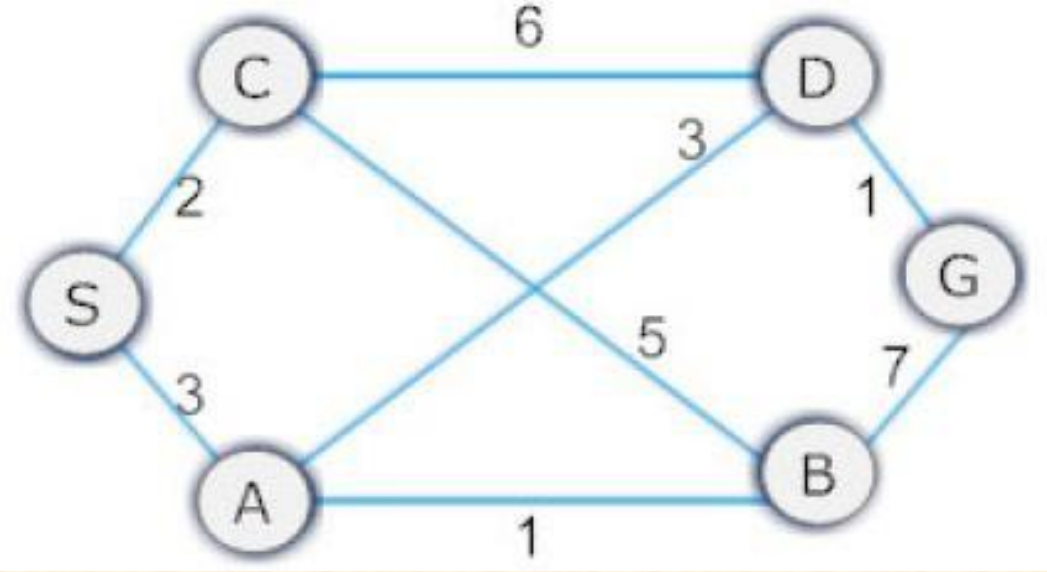
Kinerja Uninformed Search

Search method	Completeness	Time complexity	Space complexity	Optimality
Breadth first search	It is complete if b is finite	$1+b+b^2+b^3+\ldots+b^d + b(b^d-1) = O(b^d+1)$	$O(b^d+1)$ – keeps every node in memory	Optimal if all operators have the same cost. Otherwise it finds a solution with the shortest path length.
Depth first search	No: fails in infinite-depth spaces, spaces with loops	$O(b^m)$ Not a good situation when m is much higher than d. If solutions are dense, may be much faster than breadth first search.	$O(bm)$ – linear space	No
Depth limited search	Complete if solution is at depth < l	$O(b^l)$	$O(bl)$	No

Kinerja Uninformed Search

Search method	Completeness	Time complexity	Space complexity	Optimality
Iterative deepening search	Yes	$(d+1)b^0 + d\,b^1 + (d-1)b^2 + \ldots + b^d = O(b^d)$	$O(bd)$	Optimal if the cost is a constant
Uniform search	Yes	$O(b^{floor(Cost/\varepsilon)})$ where Cost is the cost of the optimal solution, ε is a positive constant and it is assumed that every step costs at lest ε. floor(x) is the largest integer not greater than *x* floor(Cost/ε) – depth of solution if all costs are approximately equal	$O(b^{floor(Cost/\varepsilon)})$	Yes
Bidirectional search	It is complete if *b* is finite	$O(b^{d/2})$	$O(b^{d/2})$	Yes

Tugas



- Mana jalur terbaik?
- Gunakan pohon (*tree*) untuk menyelesaikan masalah di atas dengan pendekatan BFS, DFS, dan DFS dengan kedalaman Iteratif.
- Buatlah program untuk mengimplementasikan pencarian BFS, DFS, DFS dengan kedalaman Iteratif dan Dua arah.